Chapter 4. Filtering

Sometimes you will want to work with every row in a table, such as:

- Purging all data from a table used to stage new data warehouse feeds
- Modifying all rows in a table after a new column has been added
- Retrieving all rows from a message queue table

In cases like these, your SQL statements won't need to have a `where` clause, since you don't need to exclude any rows from consideration. Most of the time, however, you will want to narrow your focus to a subset of a table's rows. Therefore, all the SQL data statements (except the `insert` statement) include an optional `where` clause containing one or more *filter conditions* used to restrict the number of rows acted on by the SQL statement. Additionally, the `select` statement includes a `having` clause in which filter conditions pertaining to grouped data may be included. This chapter explores the various types of filter conditions that you can employ in the `where` clauses of `select`, `update`, and `delete` statements; I demonstrate the use of filter conditions in the `having` clause of a `select` statement in Chapter 8.

Condition Evaluation

A `where` clause may contain one or more *conditions*, separated by the operators `and` and `or`. If multiple conditions are separated only by the `and` operator, then all the conditions must evaluate to `true` for the row to be included in the result set. Consider the following `where` clause:

```
WHERE first_name = 'STEVEN' AND create_date > '2006-01-01'
```

Given these two conditions, only rows where the first name is Steven and the creation date was after January 1, 2006, will be included in the result set. Though this example uses only two conditions, no matter how many conditions are in your `where` clause, if they are separated by the `and` operator, they must *all* evaluate to `true` for the row to be included in the result set.

If all conditions in the `where` clause are separated by the `or` operator, however, only *one* of the conditions must evaluate to `true` for the row to be included in the result set. Consider the following two conditions:

```
WHERE first_name = 'STEVEN' OR create_date > '2006-01-01'
```

There are now various ways for a given row to be included in the result set:

- The first name is Steven, and the creation date was after January 1, 2006.
- The first name is Steven, and the creation date was on or before January 1, 2006.
- The first name is anything other than Steven, but the creation date was after January 1, 2006.

Table 4-1 shows the possible outcomes for a `where` clause containing two conditions separated by the `or` operator.

Table 4-1. Two-condition evaluation using or

Intermediate result	Final result
`WHERE true OR true`	`true`
`WHERE true OR false`	`true`
`WHERE false OR true`	`true`
`WHERE false OR false`	`false`

In the case of the preceding example, the only way for a row to be excluded from the result set is if the person's first name was not Steven and the creation date was on or before January 1, 2006.

Using Parentheses

If your `where` clause includes three or more conditions using both the `and` and `or` operators, you should use parentheses to make your intent clear, both to the database server and to anyone else reading your code. Here's a `where` clause that extends the previous example by checking to make sure that the first name is Steven or the last name is Young, and the creation date is after January 1, 2006:

```
WHERE (first_name = 'STEVEN' OR last_name = 'YOUNG')
  AND create_date > '2006-01-01'
```

There are now three conditions; for a row to make it to the final result set, either the first *or* second condition (or both) must evaluate to `true`, and the third condition must evaluate to `true`. Table 4-2 shows the possible outcomes for this `where` clause.

Table 4-2. Three-condition evaluation using and, or

Intermediate result	Final result
`WHERE (true OR true) AND true`	`true`
`WHERE (true OR false) AND true`	`true`
`WHERE (false OR true) AND true`	`true`
`WHERE (false OR false) AND true`	`false`
`WHERE (true OR true) AND false`	`false`
`WHERE (true OR false) AND false`	`false`
`WHERE (false OR true) AND false`	`false`
`WHERE (false OR false) AND false`	`false`

As you can see, the more conditions you have in your `where` clause, the more combinations there are for the server to evaluate. In this case, only three of the eight combinations yield a final result of `true`.

Using the not Operator

Hopefully, the previous three-condition example is fairly easy to understand. Consider the following modification, however:

```
WHERE NOT (first_name = 'STEVEN' OR last_name = 'YOUNG')
  AND create_date > '2006-01-01'
```

Did you spot the change from the previous example? I added the `not` operator before the first set of conditions. Now, instead of looking for people with the first name of Steven or the last name of Young whose record was created after January 1, 2006, I am retrieving only rows where the first name is not Steven or the last name is not Young whose record was created after January 1, 2006. Table 4-3 shows the possible outcomes for this example.

Table 4-3. Three-condition evaluation using and, or, and not

Intermediate result	Final result
WHERE NOT (true OR true) AND true	false
WHERE NOT (true OR false) AND true	false
WHERE NOT (false OR true) AND true	false
WHERE NOT (false OR false) AND true	true
WHERE NOT (true OR true) AND false	false
WHERE NOT (true OR false) AND false	false
WHERE NOT (false OR true) AND false	false
WHERE NOT (false OR false) AND false	false

While it is easy for the database server to handle, it is typically difficult for a person to evaluate a `where` clause that includes the `not` operator, which is why you won't encounter it very often. In this case, you can rewrite the `where` clause to avoid using the `not` operator:

```
WHERE first_name <> 'STEVEN' AND last_name <> 'YOUNG'
  AND create_date > '2006-01-01'
```

While I'm sure that the server doesn't have a preference, you probably have an easier time understanding this version of the `where` clause.

Building a Condition

Now that you have seen how the server evaluates multiple conditions, let's take a step back and look at what comprises a single condition. A condition is made up of one or more *expressions* combined with one or more *operators*. An expression can be any of the following:

- A number
- A column in a table or view
- A string literal, such as `'Maple Street'`
- A built-in function, such as `concat('Learning', ' ', 'SQL')`
- A subquery
- A list of expressions, such as `('Boston', 'New York', 'Chicago')`

The operators used within conditions include:

- Comparison operators, such as `=`, `!=`, `<`, `>`, `<>`, `like`, `in`, and `between`
- Arithmetic operators, such as `+`, `-`, `*`, and `/`

The following section demonstrates how you can combine these expressions and operators to manufacture the various types of conditions.

Condition Types

There are many different ways to filter out unwanted data. You can look for specific values, sets of values, or ranges of values to include or exclude, or you can use various pattern-searching techniques to look for partial matches when dealing with string data. The next four subsections explore each of these condition types in detail.

Equality Conditions

A large percentage of the filter conditions that you write or come across will be of the form `'` *column* `=` *expression* `'` as in:

```
title = 'RIVER OUTLAW'
fed_id = '111-11-1111'
amount = 375.25
film_id = (SELECT film_id FROM film WHERE title = 'RIVER OUTLAW')
```

Conditions such as these are called *equality conditions* because they equate one expression to another. The first three examples equate a column to a literal (two strings and a number), and the fourth example equates a column to the value returned from a subquery. The following query uses two equality conditions, one in the `on` clause (a join condition) and the other in the `where` clause (a filter condition):

```
mysql> SELECT c.email
    -> FROM customer c
    ->   INNER JOIN rental r
    ->   ON c.customer_id = r.customer_id
    -> WHERE date(r.rental_date) = '2005-06-14';
+-------------------------------------+
| email                               |
+-------------------------------------+
| CATHERINE.CAMPBELL@sakilacustomer.org |
| JOYCE.EDWARDS@sakilacustomer.org      |
| AMBER.DIXON@sakilacustomer.org        |
| JEANETTE.GREENE@sakilacustomer.org    |
| MINNIE.ROMERO@sakilacustomer.org      |
| GWENDOLYN.MAY@sakilacustomer.org      |
| SONIA.GREGORY@sakilacustomer.org      |
| MIRIAM.MCKINNEY@sakilacustomer.org    |
| CHARLES.KOWALSKI@sakilacustomer.org   |
| DANIEL.CABRAL@sakilacustomer.org      |
| MATTHEW.MAHAN@sakilacustomer.org      |
| JEFFERY.PINSON@sakilacustomer.org     |
| HERMAN.DEVORE@sakilacustomer.org      |
| ELMER.NOE@sakilacustomer.org          |
| TERRANCE.ROUSH@sakilacustomer.org     |
| TERRENCE.GUNDERSON@sakilacustomer.org |
+-------------------------------------+
16 rows in set (0.03 sec)
```

This query shows all email addresses of every customer who rented a film on June 14, 2005.

Inequality conditions

Another fairly common type of condition is the *inequality condition*, which asserts that two expressions are *not* equal. Here's the previous query with the filter condition in the `where` clause changed to an inequality condition:

```
mysql> SELECT c.email
    -> FROM customer c
    ->   INNER JOIN rental r
    ->   ON c.customer_id = r.customer_id
    -> WHERE date(r.rental_date) <> '2005-06-14';

+---------------------------------+
| email                           |
+---------------------------------+
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
| MARY.SMITH@sakilacustomer.org   |
...
| AUSTIN.CINTRON@sakilacustomer.org |
| AUSTIN.CINTRON@sakilacustomer.org |
| AUSTIN.CINTRON@sakilacustomer.org |
| AUSTIN.CINTRON@sakilacustomer.org |
| AUSTIN.CINTRON@sakilacustomer.org |
| AUSTIN.CINTRON@sakilacustomer.org |
| AUSTIN.CINTRON@sakilacustomer.org |
| AUSTIN.CINTRON@sakilacustomer.org |
+---------------------------------+
16028 rows in set (0.03 sec)
```

This query returns all email addresses for films rented on any other date than June 14, 2005. When building inequality conditions, you may choose to use either the `!=` or `<>` operator.

Data modification using equality conditions

Equality/inequality conditions are commonly used when modifying data. For example, let's say that the movie rental company has a policy of removing old account rows once per year. Your task is to remove rows from the `rental` table where the rental date was in 2004. Here's one way to tackle it:

```
DELETE FROM rental
WHERE year(rental_date) = 2004;
```

This statement includes a single equality condition; here's an example that uses two inequality conditions to remove any rows where the rental date was not in 2005 or 2006:

```
DELETE FROM rental
WHERE year(rental_date) <> 2005 AND year(rental_date) <> 2006;
```

NOTE

When crafting examples of `delete` and `update` statements, I try to write each statement such that no rows are modified. That way, when you execute the statements, your data will remain unchanged, and your output from `select` statements will always match that shown in this book.

Since MySQL sessions are in auto-commit mode by default (see Chapter 12), you would not be able to roll back (undo) any changes made to the example data if one of my statements modified the data. You may, of course, do whatever you want with the example data, including wiping it clean and rerunning the scripts to populate the tables, but I try to leave it intact.

Range Conditions

Along with checking that an expression is equal to (or not equal to) another expression, you can build conditions that check whether an expression falls within a certain range. This type of condition is common when working with numeric or temporal data. Consider the following query:

```
mysql> SELECT customer_id, rental_date
    -> FROM rental
    -> WHERE rental_date < '2005-05-25';
+-------------+---------------------+
| customer_id | rental_date         |
+-------------+---------------------+
|         130 | 2005-05-24 22:53:30 |
|         459 | 2005-05-24 22:54:33 |
|         408 | 2005-05-24 23:03:39 |
|         333 | 2005-05-24 23:04:41 |
|         222 | 2005-05-24 23:05:21 |
|         549 | 2005-05-24 23:08:07 |
|         269 | 2005-05-24 23:11:53 |
```

```
|         239 | 2005-05-24 23:31:46 |
+-------------+---------------------+
8 rows in set (0.00 sec)
```

This query finds all film rentals prior to May 25, 2005. As well as specifying an upper limit for the rental date, you may also want to specify a lower range:

```
mysql> SELECT customer_id, rental_date
    -> FROM rental
    -> WHERE rental_date <= '2005-06-16'
    ->   AND rental_date >= '2005-06-14';
+-------------+---------------------+
| customer_id | rental_date         |
+-------------+---------------------+
|         416 | 2005-06-14 22:53:33 |
|         516 | 2005-06-14 22:55:13 |
|         239 | 2005-06-14 23:00:34 |
|         285 | 2005-06-14 23:07:08 |
|         310 | 2005-06-14 23:09:38 |
|         592 | 2005-06-14 23:12:46 |
...
|         148 | 2005-06-15 23:20:26 |
|         237 | 2005-06-15 23:36:37 |
|         155 | 2005-06-15 23:55:27 |
|         341 | 2005-06-15 23:57:20 |
|         149 | 2005-06-15 23:58:53 |
+-------------+---------------------+
364 rows in set (0.00 sec)
```

This version of the query retrieves all films rented on June 14 or 15 of 2005.

The between operator

When you have *both* an upper and lower limit for your range, you may choose to use a single condition that utilizes the `between` operator rather than using two separate conditions, as in:

```
mysql> SELECT customer_id, rental_date
    -> FROM rental
    -> WHERE rental_date BETWEEN '2005-06-14' AND '2005-06-16';
+-------------+---------------------+
| customer_id | rental_date         |
```

```
+------------+--------------------+
|        416 | 2005-06-14 22:53:33 |
|        516 | 2005-06-14 22:55:13 |
|        239 | 2005-06-14 23:00:34 |
|        285 | 2005-06-14 23:07:08 |
|        310 | 2005-06-14 23:09:38 |
|        592 | 2005-06-14 23:12:46 |
...
|        148 | 2005-06-15 23:20:26 |
|        237 | 2005-06-15 23:36:37 |
|        155 | 2005-06-15 23:55:27 |
|        341 | 2005-06-15 23:57:20 |
|        149 | 2005-06-15 23:58:53 |
+------------+--------------------+
364 rows in set (0.00 sec)
```

When using the `between` operator, there are a couple of things to keep in mind. You should always specify the lower limit of the range first (after `between`) and the upper limit of the range second (after `and`). Here's what happens if you mistakenly specify the upper limit first:

```
mysql> SELECT customer_id, rental_date
    -> FROM rental
    -> WHERE rental_date BETWEEN '2005-06-16' AND '2005-06-14';
Empty set (0.00 sec)
```

As you can see, no data is returned. This is because the server is, in effect, generating two conditions from your single condition using the `<=` and `>=` operators, as in:

```
SELECT customer_id, rental_date
    -> FROM rental
    -> WHERE rental_date >= '2005-06-16'
    ->   AND rental_date <= '2005-06-14';
Empty set (0.00 sec)
```

Since it is impossible to have a date that is *both* greater than June 16, 2005, and less than June 14, 2005, the query returns an empty set. This brings me to the second pitfall when using `between`, which is to remember that your upper and lower limits are *inclusive*, meaning that the values you provide are included in the range limits. In this case, I want to return any films rented on June 14 or 15, so I specify `2005-06-14` as the lower end of the range and `2005-06-16` as the upper end. Since I am not specifying the time

component of the date, the time defaults to midnight, so the effective range is 2005-06-14 00:00:00 to 2005-06-16 00:00:00, which will include any rentals made on June 14 or 15.

Along with dates, you can also build conditions to specify ranges of numbers. Numeric ranges are fairly easy to grasp, as demonstrated by the following:

```
mysql> SELECT customer_id, payment_date, amount
    -> FROM payment
    -> WHERE amount BETWEEN 10.0 AND 11.99;
+-------------+---------------------+--------+
| customer_id | payment_date        | amount |
+-------------+---------------------+--------+
|           2 | 2005-07-30 13:47:43 |  10.99 |
|           3 | 2005-07-27 20:23:12 |  10.99 |
|          12 | 2005-08-01 06:50:26 |  10.99 |
|          13 | 2005-07-29 22:37:41 |  11.99 |
|          21 | 2005-06-21 01:04:35 |  10.99 |
|          29 | 2005-07-09 21:55:19 |  10.99 |
...
|         571 | 2005-06-20 08:15:27 |  10.99 |
|         572 | 2005-06-17 04:05:12 |  10.99 |
|         573 | 2005-07-31 12:14:19 |  10.99 |
|         591 | 2005-07-07 20:45:51 |  11.99 |
|         592 | 2005-07-06 22:58:31 |  11.99 |
|         595 | 2005-07-31 11:51:46 |  10.99 |
+-------------+---------------------+--------+
114 rows in set (0.01 sec)
```

All payments between $10 and $11.99 are returned. Again, make sure that you specify the lower amount first.

String ranges

While ranges of dates and numbers are easy to understand, you can also build conditions that search for ranges of strings, which are a bit harder to visualize. Say, for example, you are searching for customers whose last name falls within a range. Here's a query that returns customers whose last name falls between FA and FR:

```
mysql> SELECT last_name, first_name
    -> FROM customer
    -> WHERE last_name BETWEEN 'FA' AND 'FR';
```

```
+-----------+-----------+
| last_name  | first_name |
+-----------+-----------+
| FARNSWORTH | JOHN      |
| FENNELL    | ALEXANDER  |
| FERGUSON   | BERTHA     |
| FERNANDEZ  | MELINDA    |
| FIELDS     | VICKI      |
| FISHER     | CINDY      |
| FLEMING    | MYRTLE     |
| FLETCHER   | MAE        |
| FLORES     | JULIA      |
| FORD       | CRYSTAL    |
| FORMAN     | MICHEAL    |
| FORSYTHE   | ENRIQUE    |
| FORTIER    | RAUL       |
| FORTNER    | HOWARD     |
| FOSTER     | PHYLLIS    |
| FOUST      | JACK       |
| FOWLER     | JO         |
| FOX        | HOLLY      |
+-----------+-----------+
18 rows in set (0.00 sec)
```

While there are five customers whose last name starts with FR, they are not included in the results, since a name like FRANKLIN is outside of the range. However, we can pick up four of the five customers by extending the righthand range to FRB:

```
mysql> SELECT last_name, first_name
    -> FROM customer
    -> WHERE last_name BETWEEN 'FA' AND 'FRB';
+-----------+-----------+
| last_name  | first_name |
+-----------+-----------+
| FARNSWORTH | JOHN      |
| FENNELL    | ALEXANDER  |
| FERGUSON   | BERTHA     |
| FERNANDEZ  | MELINDA    |
| FIELDS     | VICKI      |
| FISHER     | CINDY      |
| FLEMING    | MYRTLE     |
| FLETCHER   | MAE        |
| FLORES     | JULIA      |
| FORD       | CRYSTAL    |
```

```
| FORMAN     | MICHEAL    |
| FORSYTHE   | ENRIQUE    |
| FORTIER    | RAUL       |
| FORTNER    | HOWARD     |
| FOSTER     | PHYLLIS    |
| FOUST      | JACK       |
| FOWLER     | JO         |
| FOX        | HOLLY      |
| FRALEY     | JUAN       |
| FRANCISCO  | JOEL       |
| FRANKLIN   | BETH       |
| FRAZIER    | GLENDA     |
+------------+------------+
22 rows in set (0.00 sec)
```

To work with string ranges, you need to know the order of the characters within your character set (the order in which the characters within a character set are sorted is called a *collation*).

Membership Conditions

In some cases, you will not be restricting an expression to a single value or range of values but rather to a finite set of values. For example, you might want to locate all films that have a rating of either `'G'` or `'PG'`:

```
mysql> SELECT title, rating
    -> FROM film
    -> WHERE rating = 'G' OR rating = 'PG';
+--------------------------+--------+
| title                    | rating |
+--------------------------+--------+
| ACADEMY DINOSAUR         | PG     |
| ACE GOLDFINGER           | G      |
| AFFAIR PREJUDICE         | G      |
| AFRICAN EGG              | G      |
| AGENT TRUMAN             | PG     |
| ALAMO VIDEOTAPE          | G      |
| ALASKA PHANTOM           | PG     |
| ALI FOREVER              | PG     |
| AMADEUS HOLY             | PG     |
...
| WEDDING APOLLO           | PG     |
| WEREWOLF LOLA            | G      |
```

```
| WEST LION                | G     |
| WIZARD COLDBLOODED       | PG    |
| WON DARES                | PG    |
| WONDERLAND CHRISTMAS     | PG    |
| WORDS HUNTER             | PG    |
| WORST BANGER             | PG    |
| YOUNG LANGUAGE           | G     |
+--------------------------+--------+
372 rows in set (0.00 sec)
```

While this `where` clause (two conditions `or`'d together) wasn't too tedious to generate, imagine if the set of expressions contained 10 or 20 members. For these situations, you can use the `in` operator instead:

```
SELECT title, rating
FROM film
WHERE rating IN ('G','PG');
```

With the `in` operator, you can write a single condition no matter how many expressions are in the set.

Using subqueries

Along with writing your own set of expressions, such as `('G','PG')`, you can use a subquery to generate a set for you on the fly. For example, if you can assume that any film whose title includes the string `'PET'` would be safe for family viewing, you could execute a subquery against the `film` table to retrieve all ratings associated with these films and then retrieve all films having any of these ratings:

```
mysql> SELECT title, rating
    -> FROM film
    -> WHERE rating IN (SELECT rating FROM film WHERE title LIKE '%PET%');
+--------------------------+--------+
| title                    | rating |
+--------------------------+--------+
| ACADEMY DINOSAUR         | PG     |
| ACE GOLDFINGER           | G      |
| AFFAIR PREJUDICE         | G      |
| AFRICAN EGG              | G      |
| AGENT TRUMAN             | PG     |
| ALAMO VIDEOTAPE          | G      |
| ALASKA PHANTOM           | PG     |
```

```
| ALI FOREVER              | PG    |
| AMADEUS HOLY             | PG    |
...
| WEDDING APOLLO           | PG    |
| WEREWOLF LOLA            | G     |
| WEST LION                | G     |
| WIZARD COLDBLOODED       | PG    |
| WON DARES                | PG    |
| WONDERLAND CHRISTMAS     | PG    |
| WORDS HUNTER             | PG    |
| WORST BANGER             | PG    |
| YOUNG LANGUAGE           | G     |
+--------------------------+--------+
372 rows in set (0.00 sec)
```

The subquery returns the set `'G'` and `'PG'`, and the main query checks to see whether the value of the `rating` column can be found in the set returned by the subquery.

Using not in

Sometimes you want to see whether a particular expression exists within a set of expressions, and sometimes you want to see whether the expression does *not* exist within the set. For these situations, you can use the `not in` operator:

```
SELECT title, rating
FROM film
WHERE rating NOT IN ('PG-13','R', 'NC-17');
```

This query finds all accounts that are *not* rated `'PG-13'`, `'R'`, or `'NC-17'`, which will return the same set of 372 rows as the previous queries.

Matching Conditions

So far, you have been introduced to conditions that identify an exact string, a range of strings, or a set of strings; the final condition type deals with partial string matches. You may, for example, want to find all customers whose last name begins with *Q*. You could use a built-in function to strip off the first letter of the `last_name` column, as in the following:

```
mysql> SELECT last_name, first_name
    -> FROM customer
    -> WHERE left(last_name, 1) = 'Q';
+-------------+------------+
| last_name   | first_name |
+-------------+------------+
| QUALLS      | STEPHEN    |
| QUINTANILLA | ROGER      |
| QUIGLEY     | TROY       |
+-------------+------------+
3 rows in set (0.00 sec)
```

While the built-in function `left()` does the job, it doesn't give you much flexibility. Instead, you can use wildcard characters to build search expressions, as demonstrated in the next section.

Using wildcards

When searching for partial string matches, you might be interested in:

- Strings beginning/ending with a certain character
- Strings beginning/ending with a substring
- Strings containing a certain character anywhere within the string
- Strings containing a substring anywhere within the string
- Strings with a specific format, regardless of individual characters

You can build search expressions to identify these and many other partial string matches by using the wildcard characters shown in Table 4-4.

Table 4-4. Wildcard characters

Wildcard character	Matches
`_`	Exactly one character
%	Any number of characters (including 0)

The underscore character takes the place of a single character, while the percent sign can take the place of a variable number of characters. When building conditions that utilize search expressions, you use the `like` operator, as in:

```
mysql> SELECT last_name, first_name
    -> FROM customer
    -> WHERE last_name LIKE '_A_T%S';
+-----------+------------+
| last_name | first_name |
+-----------+------------+
| MATTHEWS  | ERICA      |
| WALTERS   | CASSANDRA  |
| WATTS     | SHELLY     |
+-----------+------------+
3 rows in set (0.00 sec)
```

The search expression in the previous example specifies strings containing an *A* in the second position and a *T* in the fourth position, followed by any number of characters and ending in *S*. Table 4-5 shows some more search expressions and their interpretations.

Table 4-5. Sample search expressions

Search expression	Interpretation
`F%`	Strings beginning with *F*
`%t`	Strings ending with *t*
`%bas%`	Strings containing the substring `'bas'`
`__t_`	Four-character strings with a *t* in the third position
`___-__-____`	11-character strings with dashes in the fourth and seventh positions

The wildcard characters work fine for building simple search expressions; if your needs are a bit more sophisticated, however, you can use multiple search expressions, as demonstrated by the following:

```
mysql> SELECT last_name, first_name
    -> FROM customer
    -> WHERE last_name LIKE 'Q%' OR last_name LIKE 'Y%';
+------------+------------+
| last_name  | first_name |
```

```
+------------+-----------+
| QUALLS     | STEPHEN   |
| QUIGLEY    | TROY      |
| QUINTANILLA | ROGER    |
| YANEZ      | LUIS      |
| YEE        | MARVIN    |
| YOUNG      | CYNTHIA   |
+------------+-----------+
6 rows in set (0.00 sec)
```

This query finds all customers whose last name begins with *Q* or *Y*.

Using regular expressions

If you find that the wildcard characters don't provide enough flexibility, you can use regular expressions to build search expressions. A regular expression is, in essence, a search expression on steroids. If you are new to SQL but have coded using programming languages such as Perl, then you might already be intimately familiar with regular expressions. If you have never used regular expressions, then you may want to consult Jeffrey E. F. Friedl's *Mastering Regular Expressions* (O'Reilly), since it is far too large a topic to try to cover in this book.

Here's what the previous query (find all customers whose last name starts with *Q* or *Y*) would look like using the MySQL implementation of regular expressions:

```
mysql> SELECT last_name, first_name
    -> FROM customer
    -> WHERE last_name REGEXP '^[QY]';
+------------+-----------+
| last_name  | first_name |
+------------+-----------+
| YOUNG      | CYNTHIA   |
| QUALLS     | STEPHEN   |
| QUINTANILLA | ROGER    |
| YANEZ      | LUIS      |
| YEE        | MARVIN    |
| QUIGLEY    | TROY      |
+------------+-----------+
6 rows in set (0.16 sec)
```

The `regexp` operator takes a regular expression (`'^[QY]'` in this example) and applies it to the expression on the lefthand side of the condition (the column `last_name`). The

query now contains a single condition using a regular expression rather than two conditions using wildcard characters.

Both Oracle Database and Microsoft SQL Server also support regular expressions. With Oracle Database, you would use the `regexp_like` function instead of the `regexp` operator shown in the previous example, whereas SQL Server allows regular expressions to be used with the `like` operator.

Null: That Four-Letter Word

I put it off as long as I could, but it's time to broach a topic that tends to be met with fear, uncertainty, and dread: the `null` value. `null` is the absence of a value; before an employee is terminated, for example, her `end_date` column in the `employee` table should be `null`. There is no value that can be assigned to the `end_date` column that would make sense in this situation. `null` is a bit slippery, however, as there are various flavors of `null`:

Not applicable

Such as the employee ID column for a transaction that took place at an ATM machine

Value not yet known

Such as when the federal ID is not known at the time a customer row is created

Value undefined

Such as when an account is created for a product that has not yet been added to the database

---

NOTE

Some theorists argue that there should be a different expression to cover each of these (and more) situations, but most practitioners would agree that having multiple `null` values would be far too confusing.

When working with `null`, you should remember:

- An expression can *be* null, but it can never *equal* null.
- Two nulls are never equal to each other.

To test whether an expression is `null`, you need to use the `is null` operator, as demonstrated by the following:

```
mysql> SELECT rental_id, customer_id
    -> FROM rental
    -> WHERE return_date IS NULL;
+-----------+-------------+
| rental_id | customer_id |
+-----------+-------------+
|     11496 |         155 |
|     11541 |         335 |
|     11563 |          83 |
|     11577 |         219 |
|     11593 |          99 |
...
|     15867 |         505 |
|     15875 |          41 |
|     15894 |         168 |
|     15966 |         374 |
+-----------+-------------+
183 rows in set (0.01 sec)
```

This query finds all film rentals that were never returned. Here's the same query using `= null` instead of `is null`:

```
mysql> SELECT rental_id, customer_id
    -> FROM rental
    -> WHERE return_date = NULL;
Empty set (0.01 sec)
```

As you can see, the query parses and executes but does not return any rows. This is a common mistake made by inexperienced SQL programmers, and the database server will not alert you to your error, so be careful when constructing conditions that test for `null`.

If you want to see whether a value has been assigned to a column, you can use the `is not null` operator, as in:

```
mysql> SELECT rental_id, customer_id, return_date
    -> FROM rental
    -> WHERE return_date IS NOT NULL;
```

```
+-----------+-------------+---------------------+
| rental_id | customer_id | return_date         |
+-----------+-------------+---------------------+
|         1 |         130 | 2005-05-26 22:04:30 |
|         2 |         459 | 2005-05-28 19:40:33 |
|         3 |         408 | 2005-06-01 22:12:39 |
|         4 |         333 | 2005-06-03 01:43:41 |
|         5 |         222 | 2005-06-02 04:33:21 |
|         6 |         549 | 2005-05-27 01:32:07 |
|         7 |         269 | 2005-05-29 20:34:53 |
...
|     16043 |         526 | 2005-08-31 03:09:03 |
|     16044 |         468 | 2005-08-25 04:08:39 |
|     16045 |          14 | 2005-08-25 23:54:26 |
|     16046 |          74 | 2005-08-27 18:02:47 |
|     16047 |         114 | 2005-08-25 02:48:48 |
|     16048 |         103 | 2005-08-31 21:33:07 |
|     16049 |         393 | 2005-08-30 01:01:12 |
+-----------+-------------+---------------------+
15861 rows in set (0.02 sec)
```

This version of the query returns all rentals that were returned, which is the majority of the rows in the table (15,861 out of 16,044).

Before putting `null` aside for a while, it would be helpful to investigate one more potential pitfall. Suppose that you have been asked to find all rentals that were not returned during May through August of 2005. Your first instinct might be to do the following:

```
mysql> SELECT rental_id, customer_id, return_date
    -> FROM rental
    -> WHERE return_date NOT BETWEEN '2005-05-01' AND '2005-09-01';
+-----------+-------------+---------------------+
| rental_id | customer_id | return_date         |
+-----------+-------------+---------------------+
|     15365 |         327 | 2005-09-01 03:14:17 |
|     15388 |          50 | 2005-09-01 03:50:23 |
|     15392 |         410 | 2005-09-01 01:14:15 |
|     15401 |         103 | 2005-09-01 03:44:10 |
|     15415 |         204 | 2005-09-01 02:05:56 |
...
|     15977 |         550 | 2005-09-01 22:12:10 |
|     15982 |         370 | 2005-09-01 21:51:31 |
|     16005 |         466 | 2005-09-02 02:35:22 |
|     16020 |         311 | 2005-09-01 18:17:33 |
```

```
|    16033 |        226 | 2005-09-01 02:36:15 |
|    16037 |         45 | 2005-09-01 02:48:04 |
|    16040 |        195 | 2005-09-02 02:19:33 |
+----------+------------+---------------------+
62 rows in set (0.01 sec)
```

While it is true that these 62 rentals were returned outside of the May to August window, if you look carefully at the data, you will see that all of the rows returned have a non-`null` return date. But what about the 183 rentals that were never returned? One might argue that these 183 rows were also not returned between May and August, so they should also be included in the result set. To answer the question correctly, therefore, you need to account for the possibility that some rows might contain a `null` in the `return_date` column:

```
mysql> SELECT rental_id, customer_id, return_date
    -> FROM rental
    -> WHERE return_date IS NULL
    ->   OR return_date NOT BETWEEN '2005-05-01' AND '2005-09-01';
+-----------+-------------+---------------------+
| rental_id | customer_id | return_date         |
+-----------+-------------+---------------------+
|    11496 |         155 | NULL                |
|    11541 |         335 | NULL                |
|    11563 |          83 | NULL                |
|    11577 |         219 | NULL                |
|    11593 |          99 | NULL                |
...
|    15939 |         382 | 2005-09-01 17:25:21 |
|    15942 |         210 | 2005-09-01 18:39:40 |
|    15966 |         374 | NULL                |
|    15971 |         187 | 2005-09-02 01:28:33 |
|    15973 |         343 | 2005-09-01 20:08:41 |
|    15977 |         550 | 2005-09-01 22:12:10 |
|    15982 |         370 | 2005-09-01 21:51:31 |
|    16005 |         466 | 2005-09-02 02:35:22 |
|    16020 |         311 | 2005-09-01 18:17:33 |
|    16033 |         226 | 2005-09-01 02:36:15 |
|    16037 |          45 | 2005-09-01 02:48:04 |
|    16040 |         195 | 2005-09-02 02:19:33 |
+-----------+-------------+---------------------+
245 rows in set (0.01 sec)
```

The result set now includes the 62 rentals that were returned outside of the May to August window, along with the 183 rentals that were never returned, for a total of 245 rows. When working with a database that you are not familiar with, it is a good idea to find out which columns in a table allow `null`s so that you can take appropriate measures with your filter conditions to keep data from slipping through the cracks.

Test Your Knowledge

The following exercises test your understanding of filter conditions. Please see Appendix B for solutions.

You'll need to refer to the following subset of rows from the `payment` table for the first two exercises:

```
+------------+-------------+--------+--------------------+
| payment_id | customer_id | amount | date(payment_date) |
+------------+-------------+--------+--------------------+
|        101 |           4 |   8.99 | 2005-08-18         |
|        102 |           4 |   1.99 | 2005-08-19         |
|        103 |           4 |   2.99 | 2005-08-20         |
|        104 |           4 |   6.99 | 2005-08-20         |
|        105 |           4 |   4.99 | 2005-08-21         |
|        106 |           4 |   2.99 | 2005-08-22         |
|        107 |           4 |   1.99 | 2005-08-23         |
|        108 |           5 |   0.99 | 2005-05-29         |
|        109 |           5 |   6.99 | 2005-05-31         |
|        110 |           5 |   1.99 | 2005-05-31         |
|        111 |           5 |   3.99 | 2005-06-15         |
|        112 |           5 |   2.99 | 2005-06-16         |
|        113 |           5 |   4.99 | 2005-06-17         |
|        114 |           5 |   2.99 | 2005-06-19         |
|        115 |           5 |   4.99 | 2005-06-20         |
|        116 |           5 |   4.99 | 2005-07-06         |
|        117 |           5 |   2.99 | 2005-07-08         |
|        118 |           5 |   4.99 | 2005-07-09         |
|        119 |           5 |   5.99 | 2005-07-09         |
|        120 |           5 |   1.99 | 2005-07-09         |
+------------+-------------+--------+--------------------+
```

Exercise 4-1

Which of the payment IDs would be returned by the following filter conditions?

```
customer_id <> 5 AND (amount > 8 OR date(payment_date) = '2005-08-23')
```

Exercise 4-2

Which of the payment IDs would be returned by the following filter conditions?

```
customer_id = 5 AND NOT (amount > 6 OR date(payment_date) = '2005-06-19')
```

Exercise 4-3

Construct a query that retrieves all rows from the `payments` table where the amount is either 1.98, 7.98, or 9.98.

Exercise 4-4

Construct a query that finds all customers whose last name contains an *A* in the second position and a *W* anywhere after the *A*.

Previous chapter
< 3. Query Primer

Next chapter
5. Querying Multiple Tables >